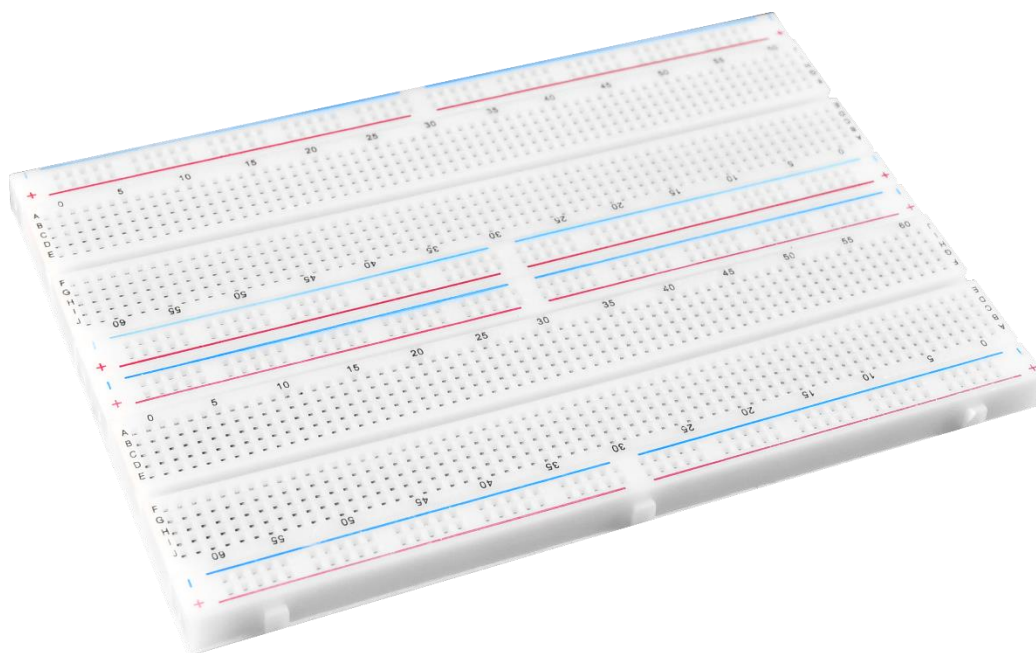


面包板实验手册 V1.0



正点原子

广州市星翼电子科技有限公司

修订历史:

版本	日期	作者	修改内容
V1.0	20230911	徐伟健	第一次发布



正点原子公司名称 : 广州市星翼电子科技有限公司
原子哥在线教学平台 : www.yuanzige.com
开源电子网 / 论坛 : <http://www.openedv.com/forum.php>
正点原子官方网站 : www.alientek.com
正点原子淘宝店铺 : <https://openedv.taobao.com>
正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请下载原子哥 APP, 数千讲视频免费学习, 更快更流畅。
请关注正点原子公众号, 资料发布更新我们会通知。



扫码下载“原子哥”APP



扫码关注正点原子公众号

内容简介.....	1
第一章 LED 闪烁实验	2
1.1 硬件设计.....	2
1.1.1 例程功能.....	2
1.1.2 硬件资源.....	2
1.1.3 原理图.....	2
1.2 程序设计.....	3
1.2.1 HAL 库的 GPIO 函数.....	3
1.2.2 LED 驱动配置流程	4
1.2.3 LED 驱动代码	4
1.2.3 实验应用代码.....	5
1.3 下载验证.....	6
第二章 LED 跑马灯实验	7
2.1 硬件设计.....	7
2.1.1 例程功能.....	7
2.1.2 硬件资源.....	7
2.1.3 原理图.....	7
2.2 程序设计.....	8
2.2.1 LED 驱动代码	8
2.2.2 实验应用代码.....	9
2.3 下载验证.....	10
第三章 单个按键控制 LED 实验	11
3.1 硬件设计.....	11
3.1.1 例程功能.....	11
3.1.2 硬件资源.....	11
3.1.3 原理图.....	11
3.2 程序设计.....	12
3.2.1 HAL 库的 GPIO 函数.....	12
3.2.2 KEY 驱动配置流程.....	12
3.2.3 按键驱动代码.....	13
3.2.4 实验应用代码.....	13
3.3 下载验证.....	14
第四章 多个按键控制 LED 实验	15
4.1 硬件设计.....	15
4.1.1 例程功能.....	15
4.1.2 硬件资源.....	15
4.1.3 原理图.....	15
4.2 程序设计.....	16
4.2.3 按键驱动代码.....	16
4.2.4 实验应用代码.....	17
4.3 下载验证.....	18
第五章 PWM 控制 LED 实验.....	19
5.1 硬件设计.....	19
5.1.1 例程功能.....	19

5.1.2 硬件资源.....	19
5.1.3 原理图.....	19
5.2 程序设计.....	19
5.2.1 HAL 库的 TIM 函数.....	19
5.2.2 TIM 的 PWM 输出驱动配置流程	21
5.2.3 TIM 输出 PWM 驱动代码	21
5.2.4 实验应用代码.....	23
5.3 下载验证.....	24
第六章 可调亮度 LED 实验	26
6.1 硬件设计.....	26
6.1.1 例程功能.....	26
6.1.2 硬件资源.....	26
6.1.3 原理图.....	26
6.2 程序设计.....	27
6.2.1 HAL 库 ADC 驱动函数	27
6.2.2 ADC 配置流程.....	29
6.2.3 ADC 驱动代码.....	29
6.2.4 实验应用代码.....	31
6.3 下载验证.....	32

内容简介

本手册主要介绍正点原子实验箱面包板配件上的基础使用教程，围绕正点原子的 STM32F103C8T6 小系统板进行设计。本手册带领大家学会面包板上的快速开发。

本手册共有六个实验：

- 1、LED 闪烁实验
- 2、LED 跑马灯实验
- 3、单个按键控制 LED 实验
- 4、多个按键控制 LED 实验
- 5、PWM 控制 LED 实验
- 6、可调亮度 LED 实验

本手册为正点原子嵌入式学习实验箱的配套教程。在配套资料包中，除了提供面包板实验手册中涉及的源码，还会提供大量与本板卡相关的资料，包括硬件原理图以及小系统板的所有实验例程的源代码，这些代码都经过了详细的注释，并且都经过了测试，不会出现任何错误或警告。

本手册非常适合电子工程专业学生使用，由浅入深的实验安排，大大提高大家的动手积极性，快速入门嵌入式开发。

第一章 LED 闪烁实验

本章将介绍嵌入式开发中经典的“跑马灯”程序。该程序在嵌入式的地位就相当于 C 语言中的 Hello_World，打开嵌入式开发大门。程序的本质是通过控制单片机的 GPIO（通用输入/输出）输出高低电平，控制 LED0（发光二极管）的亮灭状态进行切换。通过本章的学习，读者将了解 GPIO 输出模式的使用及其在嵌入式开发中的应用。

本章分为如下几个小节：

- 1.1 硬件设计
- 1.2 程序设计
- 1.3 下载验证

1.1 硬件设计

1.1.1 例程功能

LED0 以 500 毫秒的频率交替闪烁，实现类似跑马灯的效果。

1.1.2 硬件资源

- 1. LED
- LED0 – PA8

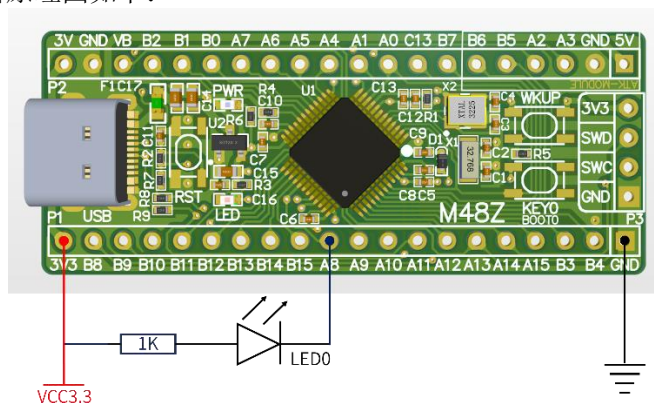
1.1.3 原理图

本章实验所使用 LED 并不是最小系统板载的贴片 LED，而是配件包里面的直插式 LED。当把 VCC 接去 LED 的正极，把 GND 接去 LED 的负极，LED 就会亮起来。对于直插式 LED，这个正负极主要体现在引脚的长度，长脚为正极，短脚为负极，如下图所示。



1.1.3.1 直插式 LED 实物图

本章设计的电路原理图如下：



1.1.3.2 LED0 与 MCU 的连接原理图

根据上述原理图，LED0 的正极通过一个限流电阻连接到电源正极，而负极则与 MCU 的 PA8 引脚相连接。因此，通过控制 PA8 引脚输出电平来控制 LED0 的亮灭。如果 PA8 引脚输出高电平，则 LED0 将熄灭。如果 PA8 引脚输出低电平，则 LED0 将点亮。

基于面包板上的实物连接图，可以查看下载验证中的效果图。

1.2 程序设计

1.2.1 HAL 库的 GPIO 函数

以下是本实验使用的 HAL 库 GPIO 函数的常用函数，这些函数用于控制 GPIO 引脚的状态，例如设置引脚的输入或输出模式、设置引脚的电平等。

1. 配置 GPIO 函数

该函数用于配置 GPIO 引脚功能和各项参数，其函数原型如下：

```
void HAL_GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_Init)
```

此函数的形参描述，如下表所示：

形参	描述
GPIOx	指向 GPIO 端口结构体的指针 例如：GPIOA、GPIOB 等（在 stm32f103xe.h 文件中有定义）
GPIO_Init	指向 GPIO 初始化结构体的指针 需自行定义，并根据 GPIO 的配置参数填充结构体中的成员变量

表 1.2.1.1 函数 HAL_GPIO_Init() 形参描述

返回值：无。

该函数使用 GPIO_InitTypeDef 类型的结构体变量作为参数，以配置 GPIO 引脚。该结构体的定义如下：

```
typedef struct
{
    uint32_t Pin;           /* 指定要配置的 GPIO 引脚 */
    uint32_t Mode;          /* 模式 */
    uint32_t Pull;          /* 上拉/下拉 */
    uint32_t Speed;         /* 速度 */
} GPIO_InitTypeDef;

/* Pin */
#define GPIO_PIN_0          ((uint16_t)0x0001)
#define GPIO_PIN_1          ((uint16_t)0x0002)
#define GPIO_PIN_2          ((uint16_t)0x0004)
#define GPIO_PIN_3          ((uint16_t)0x0008)
#define GPIO_PIN_4          ((uint16_t)0x0010)
#define GPIO_PIN_5          ((uint16_t)0x0020)
#define GPIO_PIN_6          ((uint16_t)0x0040)
#define GPIO_PIN_7          ((uint16_t)0x0080)
#define GPIO_PIN_8          ((uint16_t)0x0100)
#define GPIO_PIN_9          ((uint16_t)0x0200)
#define GPIO_PIN_10         ((uint16_t)0x0400)
#define GPIO_PIN_11         ((uint16_t)0x0800)
#define GPIO_PIN_12         ((uint16_t)0x1000)
#define GPIO_PIN_13         ((uint16_t)0x2000)
#define GPIO_PIN_14         ((uint16_t)0x4000)
#define GPIO_PIN_15         ((uint16_t)0x8000)
#define GPIO_PIN_All        ((uint16_t)0xFFFF)

/* Mode */
#define GPIO_MODE_INPUT      0x00000000u    /* 输入模式 */
#define GPIO_MODE_OUTPUT_PP  0x00000001u    /* 通用输出模式 */
#define GPIO_MODE_AF_PP     0x00000002u    /* 复用功能模式 */
#define GPIO_MODE_ANALOG    0x00000003u    /* 模拟模式 */

/* Pull */
#define GPIO_NOPULL          0x00000000u    /* 禁止上拉/下拉 */
#define GPIO_PULLUP          0x00000001u    /* 上拉 */
#define GPIO_PULLDOWN        0x00000002u    /* 下拉 */
```

```
/* Speed */
#define GPIO_SPEED_FREQ_LOW          (GPIO_CRL_MODE0_1) /* 低速 */
#define GPIO_SPEED_FREQ_MEDIUM       (GPIO_CRL_MODE0_0) /* 中速 */
#define GPIO_SPEED_FREQ_HIGH         (GPIO_CRL_MODE0)   /* 高速 */
```

2. 设置 GPIO 的电平函数

此函数用于设置 GPIO 引脚输出指定电平（高电平或低电平），其函数原型如下

```
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx,
                       uint16_t GPIO_Pin,
                       GPIO_PinState PinState);
```

该函数的形参描述，如下表所示：

形参	描述
GPIOx	指向 GPIO 端口结构体的指针 例如：GPIOA、GPIOB 等（在 stm32f103xe.h 文件中有定义）
GPIO_Pin	待设置的 GPIO 引脚号 如：GPIO_PIN_0 ~ GPIO_PIN_15
PinState	引脚状态 GPIO_PIN_RESET: 0 GPIO_PIN_SET: 1

表 1.2.1.2 函数 HAL_GPIO_WritePin()形参描述

返回值：无。

3. GPIO 翻转电平函数

此函数用于翻转 GPIO 引脚的输出电平，其函数原型如下：

```
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
```

该函数的形参描述，如下表所示：

形参	描述
GPIOx	指向 GPIO 端口结构体的指针 例如：GPIOA、GPIOB 等（在 stm32f103xe.h 文件中有定义）
GPIO_Pin	待设置的 GPIO 引脚号 如：GPIO_PIN_0 ~ GPIO_PIN_15

表 1.2.1.3 函数 HAL_GPIO_TogglePin()形参描述

返回值：无。

1.2.2 LED 驱动配置流程

控制 GPIO 引脚输出高低电平来控制 LED 等亮灭状态需要进行以下两个步骤：

1. 配置 GPIO 引脚的模式，即输出模式
2. 配置 GPIO 引脚输出的电平

为了实现这两个步骤，需要使用 HAL 库中的驱动函数。这些驱动函数请见上小节。

1.2.3 LED 驱动代码

LED 驱动编写流程一般包括以下几个步骤：

1. led.c/h 文件（在工程中创建文件非常简单，这里无需讲解）
2. 声明引脚定义和操作引脚定义
3. 调用 HAL 库提供的 API 函数，编写 led_init 初始化函数

LED 驱动中，声明引脚的定义，如下所示：

```
#define LED0_GPIO_PORT      GPIOA
#define LED0_GPIO_PIN       GPIO_PIN_8
#define LED0_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)
```

在上述的宏定义中，GPIO 端口、GPIO 引脚号、GPIO 端口时钟使能和 GPIO 复用功能等信息被定义。这些宏定义将用于后续的实验代码中，以定义 GPIO 引脚的各种信息

LED 驱动中，操作引脚的定义，如下所示：

```
/* LED 端口定义 */
```

```
#define LED0(x)    do{ x ? \
    HAL_GPIO_WritePin(LED0_GPIO_PORT, LED0_GPIO_PIN, GPIO_PIN_SET) : \
    HAL_GPIO_WritePin(LED0_GPIO_PORT, LED0_GPIO_PIN, GPIO_PIN_RESET); \
    }while(0)      /* LED0 翻转 */

/* LED 取反定义 */
#define LED0_TOGGLE()
do{ HAL_GPIO_TogglePin(LED0_GPIO_PORT, LED0_GPIO_PIN); }while(0)
```

在上述的宏定义中，包含了对 GPIO 引脚的各种操作定义。这些定义将用于后续的实验代码中，以定义 GPIO 引脚的各种操作。

LED 驱动中，LED 的初始化函数，如下所示：

```
/**
 * @brief      初始化 LED 相关 IO 口，并使能时钟
 * @param      无
 * @retval     无
 */
void led_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    LED0_GPIO_CLK_ENABLE();          /* LED0 时钟使能 */

    gpio_init_struct.Pin = LED0_GPIO_PIN;          /* LED0 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;    /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;           /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH; /* 高速 */
    HAL_GPIO_Init(LED0_GPIO_PORT, &gpio_init_struct); /* 初始化 LED0 引脚 */

    LED0(1);          /* 关闭 LED0 */
}
```

在 LED 的初始化函数中，开发者使能了 LED0 控制引脚的 GPIO 端口时钟，并将其配置为通用输出模式。此外，初始化函数还默认将 LED 状态设置为熄灭状态。

1.2.3 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    HAL_Init();          /* 初始化 HAL 库 */
    sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟，72Mhz */
    delay_init(72);      /* 延时初始化 */
    usart_init(115200);  /* 初始化串口 */
    led_init();          /* 初始化 LED */

    while (1)
    {
        LED0(0);          /* LED0 亮 */
        delay_ms(500);
        LED0(1);          /* LED0 灭 */
        delay_ms(500);
    }
}
```

可以看到，在调用 LED 初始化之前，会先调用以下函数：

```
sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟，72Mhz */
delay_init(72);          /* 延时初始化 */
usart_init(115200);      /* 初始化串口 */
```

①：第一行代码用于配置系统时钟，由于大部分实验例程都无需考虑低功耗应用，因此它们配置系统时钟的参数与本实验相同，即将系统主频配置为 72MHz。

②：第二行代码用于初始化延时功能，为后续应用代码提供微秒级和毫秒级的延时功能。

③：第三行代码用于初始化用于调试功能的 USART1，为后续应用代码提供 printf 等功能，方便通过串口调试助手查看程序运行情况。

以上三行代码是执行一些必要的初始化，这些都是在后续每个实验例程中都会见到的，当有 IO 冲突的时候，也有可能注释掉 USART1 的初始化。

在程序执行完必要的初始化操作后，系统会紧接着初始化 LED，并将其设置为输出状态。接着，在一个 while 循环中，系统会重复地控制板载 LED0 亮起和熄灭，轮流时间为 500 毫秒，以此实现跑马灯的效果。

1.3 下载验证

在完成编译和烧录操作后，运行结果如下 1.3.1 所示。

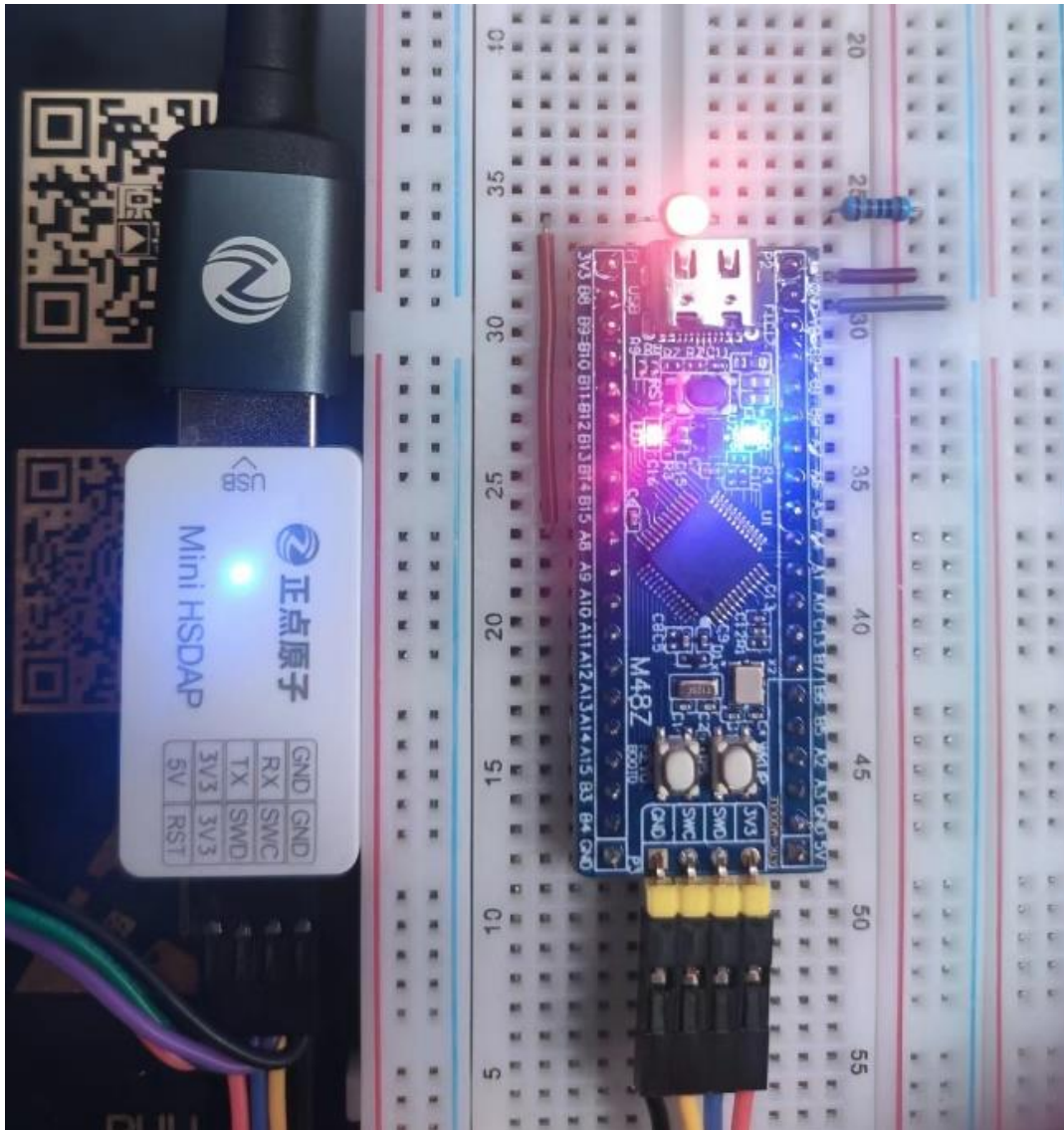


图 1.3.1 程序运行结果

我们可以观察到板子上的 LED0 轮流亮起和熄灭，轮流的时间大约为 500 毫秒，符合程序设定的实验效果，表明程序已经成功编译和烧录到开发板上。

第二章 LED 跑马灯实验

在之前的章节中，我们只是点亮一个 LED，在实际的应用中，可能还需要控制多个 LED 的亮灭，所以这里我们进行 4 个 LED 灯的控制，实现跑马灯效果，继续巩固 GPIO 作为输出功能这个知识点。

本章分为如下几个小节：

- 2.1 硬件设计
- 2.2 程序设计
- 2.3 下载验证

2.1 硬件设计

2.1.1 例程功能

在第一章的基础上，新增 3 个 LED。4 个 LED 有顺序的亮起来，灭下去，实现流水灯的效果。

2.1.2 硬件资源

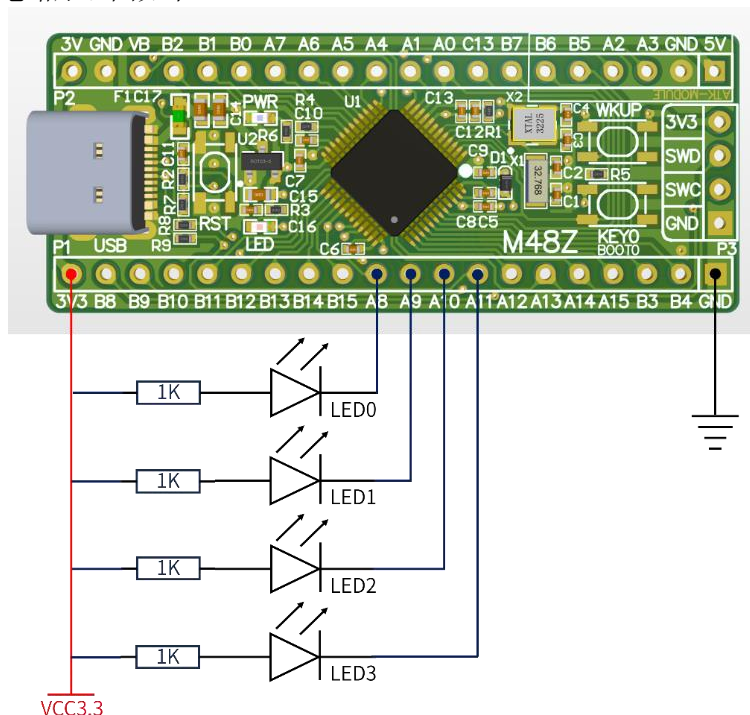
1. LED

LED0 – PA8 LED1 – PA9 LED2 – PA10 LED3 – PA11

2.1.3 原理图

本章相比上一章新增的三个 LED（LED1、LED2 和 LED3），这三个 LED 的连接方式跟 LED0 是一样的，也是正极通过一个 1K 的限流电阻连接到电源正极，而负极则与 MCU 的 GPIO 相连接。

本章设计的电路原理图如下：



2.1.3.1 LED 与 MCU 的连接原理图

基于面包板上的实物连接图，可以查看下载验证中的效果图。

2.2 程序设计

本实验使用的 HAL 库 GPIO 函数跟上一个实验是一样的。对比上一个实验只是添加多了 3 个 LED，没有添加别的东西，所以在 LED 驱动配置流程也是一样的，配置 GPIO 引脚的模式以及配置 GPIO 引脚输出的电平。

2.2.1 LED 驱动代码

本实验的 LED 驱动会在第一个实验的 LED 驱动上添加三个 LED，添加方式也是跟 LED0 是一致的，这里就不再细说。

在 led.h 文件中，共定义了 4 个 LED 相关的宏，如下图所示：

```
/* 引脚定义 */
#define LED0_GPIO_PORT      GPIOA
#define LED0_GPIO_PIN      GPIO_PIN_8
#define LED0_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define LED1_GPIO_PORT      GPIOA
#define LED1_GPIO_PIN      GPIO_PIN_9
#define LED1_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define LED2_GPIO_PORT      GPIOA
#define LED2_GPIO_PIN      GPIO_PIN_10
#define LED2_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define LED3_GPIO_PORT      GPIOA
#define LED3_GPIO_PIN      GPIO_PIN_11
#define LED3_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)
```

操作引脚的宏函数也定义了 4 个，如下所示：

```
/* LED 端口定义 */
#define LED0(x) do{ x ? \
    HAL_GPIO_WritePin(LED0_GPIO_PORT, LED0_GPIO_PIN, GPIO_PIN_SET) : \
    HAL_GPIO_WritePin(LED0_GPIO_PORT, LED0_GPIO_PIN, GPIO_PIN_RESET); \
}while(0) /* LED0 翻转 */

/* LED 取反定义 */
#define LED0_TOGGLE() do{ HAL_GPIO_TogglePin(LED0_GPIO_PORT, LED0_GPIO_PIN); }while(0)

#define LED1(x) do{ x ? \
    HAL_GPIO_WritePin(LED1_GPIO_PORT, LED1_GPIO_PIN, GPIO_PIN_SET) : \
    HAL_GPIO_WritePin(LED1_GPIO_PORT, LED1_GPIO_PIN, GPIO_PIN_RESET); \
}while(0)

#define LED1_TOGGLE() do{ HAL_GPIO_TogglePin(LED1_GPIO_PORT, LED1_GPIO_PIN); }while(0)

#define LED2(x) do{ x ? \
    HAL_GPIO_WritePin(LED2_GPIO_PORT, LED2_GPIO_PIN, GPIO_PIN_SET) : \
    HAL_GPIO_WritePin(LED2_GPIO_PORT, LED2_GPIO_PIN, GPIO_PIN_RESET); \
}while(0)

#define LED2_TOGGLE() do{ HAL_GPIO_TogglePin(LED2_GPIO_PORT, LED2_GPIO_PIN); }while(0)

#define LED3(x) do{ x ? \
    HAL_GPIO_WritePin(LED3_GPIO_PORT, LED3_GPIO_PIN, GPIO_PIN_SET) : \
    HAL_GPIO_WritePin(LED3_GPIO_PORT, LED3_GPIO_PIN, GPIO_PIN_RESET); \
}while(0)

#define LED3_TOGGLE() do{ HAL_GPIO_TogglePin(LED3_GPIO_PORT, LED3_GPIO_PIN); }while(0)
```

LED 的初始化函数除了要初始化 LED0，还需要初始化 LED1、LED2 和 LED3，如下所示：

```

/**
 * @brief      初始化 LED 相关 IO 口，并使能时钟
 * @param      无
 * @retval     无
 */
void led_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    LED0_GPIO_CLK_ENABLE();           /* LED0 时钟使能 */
    LED1_GPIO_CLK_ENABLE();           /* LED1 时钟使能 */
    LED2_GPIO_CLK_ENABLE();           /* LED2 时钟使能 */
    LED3_GPIO_CLK_ENABLE();           /* LED3 时钟使能 */

    gpio_init_struct.Pin = LED0_GPIO_PIN;           /* LED0 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;     /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;             /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;   /* 高速 */
    HAL_GPIO_Init(LED0_GPIO_PORT, &gpio_init_struct); /* 初始化 LED0 引脚 */

    gpio_init_struct.Pin = LED1_GPIO_PIN;           /* LED1 引脚 */
    HAL_GPIO_Init(LED1_GPIO_PORT, &gpio_init_struct); /* 初始化 LED1 引脚 */

    gpio_init_struct.Pin = LED2_GPIO_PIN;           /* LED2 引脚 */
    HAL_GPIO_Init(LED2_GPIO_PORT, &gpio_init_struct); /* 初始化 LED2 引脚 */

    gpio_init_struct.Pin = LED3_GPIO_PIN;           /* LED3 引脚 */
    HAL_GPIO_Init(LED3_GPIO_PORT, &gpio_init_struct); /* 初始化 LED3 引脚 */

    LED0(1);           /* 关闭 LED0 */
    LED1(1);           /* 关闭 LED1 */
    LED2(1);           /* 关闭 LED2 */
    LED3(1);           /* 关闭 LED3 */
}

```

在 LED 的初始化函数中，开发者使能了 LED0/1/2/3 控制引脚的 GPIO 端口时钟，并将其配置为通用输出模式。此外，初始化函数还默认将 4 个 LED 状态设置为熄灭状态。

2.2.2 实验应用代码

本章实验的应用代码，如下所示：

```

#include "SYSTEM/sys/sys.h"
#include "SYSTEM/delay/delay.h"
#include "SYSTEM/usart/usart.h"
#include "BSP/LED/led.h"

int main(void)
{
    uint8_t key = 0;

    HAL_Init();           /* 初始化 HAL 库 */
    sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟，72Mhz */
    delay_init(72);       /* 延时初始化 */
    usart_init(115200);    /* 初始化串口 */
    led_init();           /* 初始化 LED */

    while (1)
    {
        LED0(0);          /* LED0 亮 */
        delay_ms(500);
        LED1(0);          /* LED1 亮 */
        delay_ms(500);
        LED2(0);          /* LED2 亮 */
    }
}

```

```
delay_ms(500);  
LED3(0);          /* LED3 亮 */  
delay_ms(500);  
LED3(1);          /* LED3 灭 */  
delay_ms(500);  
LED2(1);          /* LED2 灭 */  
delay_ms(500);  
LED1(1);          /* LED1 灭 */  
delay_ms(500);  
LED0(1);          /* LED0 灭 */  
delay_ms(500);  
}  
}
```

在应用代码中，初始化完 LED 后，就进入了一个无限循环中。在循环中，LED0 向 LED3 按顺序逐个亮起，然后又从 LED3 开始向 LED1 逐个顺序熄灭。

2.3 下载验证

在完成编译和烧录操作后，运行结果如下 2.3.1 所示。

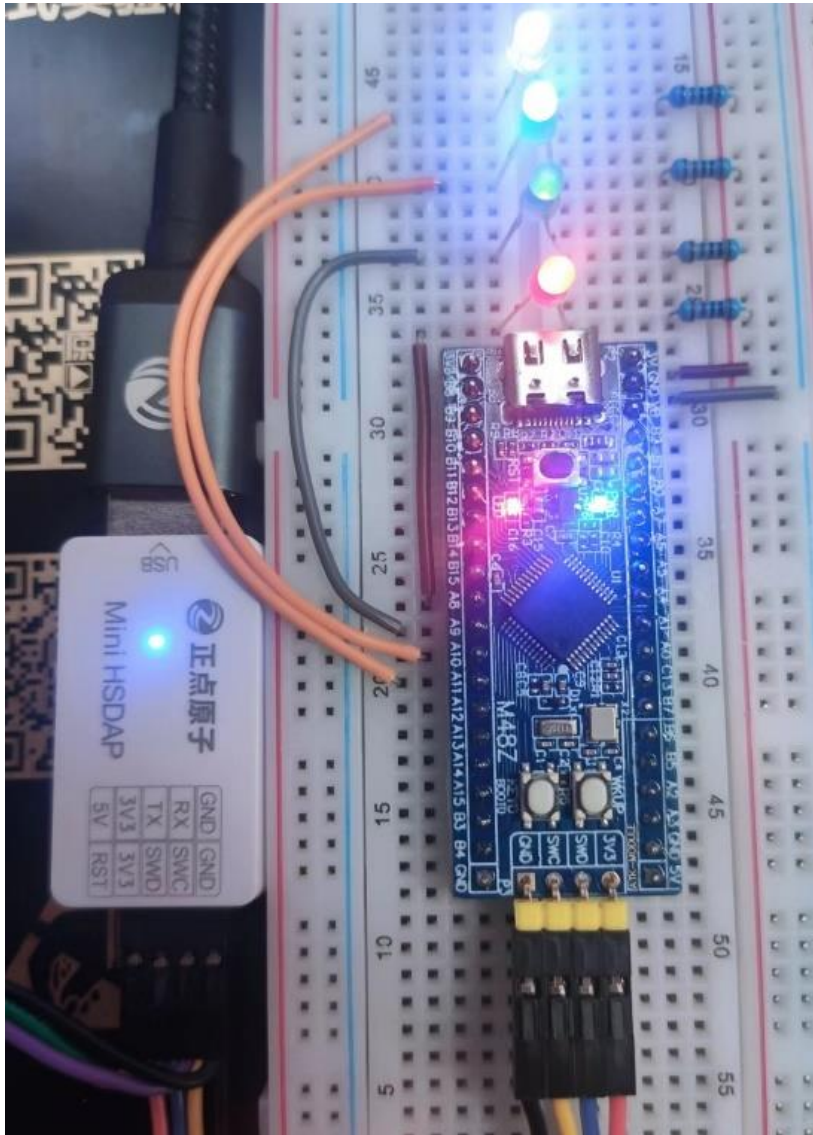


图 2.3.1 程序运行结果

我们会观察到 4 个 LED 会轮流亮起，轮流熄灭，实现了跑马灯效果。

第三章 单个按键控制 LED 实验

在之前的章节中，我们介绍了 GPIO 的输出模式，包括如何使用 GPIO 口来控制 LED 的亮灭。在实际的应用中，我们还需要使用 GPIO 的输入模式来获取外部的输入信号，例如获取按键的状态。在本章节中，我们将介绍 GPIO 作为输入模式的使用。

本章分为如下几个小节：

- 3.1 硬件设计
- 3.2 程序设计
- 3.3 下载验证

3.1 硬件设计

3.1.1 例程功能

通过按下 KEY0 按键，可以控制 LED0 点亮。这个控制是通过在控制电路中读取按键输入的信号来实现的。

3.1.2 硬件资源

- 1. LED
LED0 – PA8
- 2. 按键
KEY0 – PB0

3.1.3 原理图

本章实验使用到的 KEY 并非小系统板的板载的贴片按键，而是配件包里面的立式按键。实物图如下：



图 3.1.3.1 立式按键实物图

当我们按下时，上图按键背面的红色两端就会导通，松手就会断开。这种按键会存在抖动问题，因为按键内部使用的是机械式弹簧片来进行通断的，在按下和松开瞬间会伴随一连串的抖动，如下图所示：

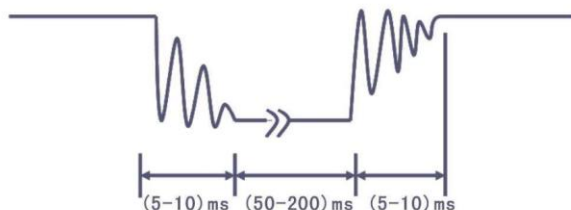


图 3.1.3.2 按键抖动图

由于存在按键抖动问题，所以我们在检测按键是否被按下时，需要加入延时过滤抖动时间。接下来，我们看一下本章设计的电路原理图如下：

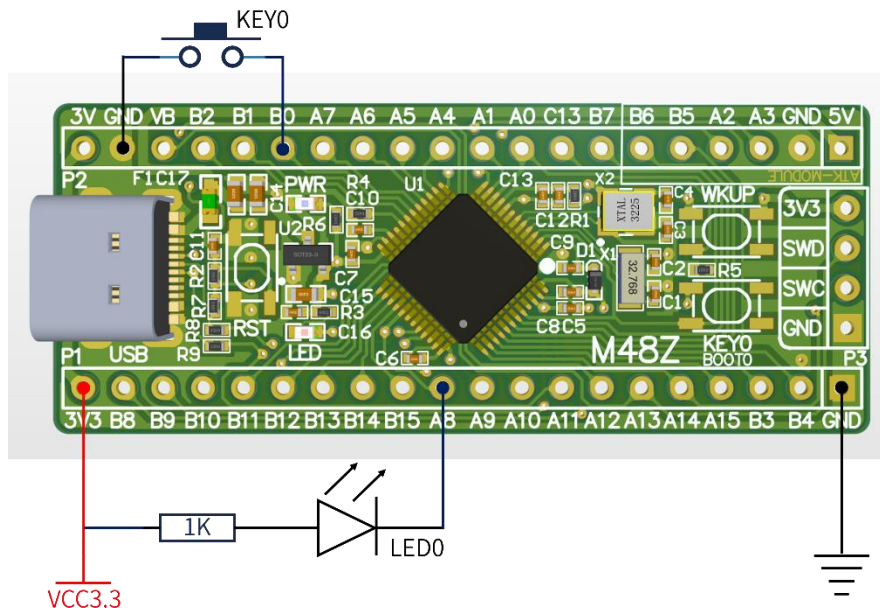


图 3.1.3.3 按键与 MCU 的连接原理图

从上面的原理图中可以看出，KEY0 按键的一端连接到电源负极，而另一端连接到 MCU 的 PB0 引脚。当 KEY0 被按下时，MCU 对应的引脚都能够读取到低电平的状态。当松开按键后，MCU 对应的引脚读取到的电平状态是确定的。

为了读取 KEY0 按键的状态，它的引脚需要配置为输入模式，并且需要配置成上拉，这样当引脚悬空时，它会被上拉到电源正极，从而保证引脚电平的准确性。在实际应用中，需要根据电路设计的需求，对按键和 LED 的接线方式进行合理配置。

基于面包板上的实物连接图，可以查看下载验证中的效果图。

3.2 程序设计

3.2.1 HAL 库的 GPIO 函数

以下是本实验使用的 HAL 库 GPIO 函数的常用函数：

读取 GPIO 引脚的输入电平函数

该函数用于读取 GPIO 引脚的输入电平（高电平或低电平），其函数原型如下所示：

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
```

该函数的形参描述，如下表所示

形参	描述
GPIOx	指向 GPIO 端口结构体的指针 例如：GPIOA、GPIOB...
GPIO_Pin	待设置的 GPIO 引脚号 如：GPIO_PIN_0 ~ GPIO_PIN_15

表 3.2.1.1 函数 GPIO_Input_Pin_Data_Get()形参描述

返回值：输入引脚的电平。

3.2.2 KEY 驱动配置流程

在本章实验中，要通过读取 GPIO 引脚的输入状态来判断按键是否被按下，以此来控制对应 LED 的状态翻转。为此，需要执行以下步骤：

1. 将 GPIO 引脚配置为输入模式和下拉，请见 3.2.1 小节中的配置 GPIO 引脚的相关内容
2. 读取 GPIO 引脚的输入状态

为了实现这两个步骤，需要使用 HAL 库中的驱动函数。这些驱动函数请见上小节。

3.2.3 按键驱动代码

KEY 驱动编写流程一般包括以下几个步骤：

1. 创建 key.c/h 文件（在工程中创建文件非常简单，这里无需讲解）
2. 声明引脚定义和操作引脚定义
3. 调用 HAL 库提供的 API 函数，编写 key_init 函数

KEY 驱动中，声明引脚的定义，如下所示：

```
/* 引脚定义 */
#define KEY0_GPIO_PORT      GPIOB
#define KEY0_GPIO_PIN      GPIO_PIN_0
#define KEY0_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)
```

KEY 驱动中，操作引脚的定义，如下所示：

```
/* I/O 操作 */
#define KEY0      HAL_GPIO_ReadPin(KEY0_GPIO_PORT, KEY0_GPIO_PIN) /* 读取 KEY0 */
```

KEY 驱动中，按键的初始化函数，如下所示：

```
/**
 * @brief      按键初始化函数
 * @param      无
 * @retval     无
 */
void key_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    KEY0_GPIO_CLK_ENABLE(); /* KEY0 时钟使能 */

    gpio_init_struct.Pin = KEY0_GPIO_PIN; /* KEY0 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_INPUT; /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLUP; /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH; /* 高速 */
    HAL_GPIO_Init(KEY0_GPIO_PORT, &gpio_init_struct);
}
```

在按键初始化函数中，对于 KEY0 按键对应的 GPIO 引脚，开启了其 GPIO 端口时钟，并将其设置为输入模式和上拉模式。

3.2.4 实验应用代码

本章实验的应用代码，如下所示：

```
#include " ./SYSTEM/sys/sys.h"
#include " ./SYSTEM/delay/delay.h"
#include " ./SYSTEM/usart/usart.h"
#include " ./BSP/LED/led.h"
#include " ./BSP/KEY/key.h"

int main(void)
{
    uint8_t key = 0;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟，72Mhz */
    delay_init(72); /* 延时初始化 */
    usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */

    while (1)
    {
        if (KEY0 == 0) /* 按键被按下 */
        {
```

```
delay_ms(10); /* 按键消抖 */  
  
if (KEY0 == 0)  
{  
    LED0(0);  
}  
}  
else  
{  
    LED0(1);  
}  
}  
}
```

在应用代码中，初始化完 LED 和 KEY 后，就进入了一个无限循环中。在循环中来判断按键的状态。如果扫描到 KEY0 按键被按下，LED0 就会被点亮，否则是熄灭状态。

3.3 下载验证

在完成编译和烧录操作后，运行结果如下 3.3.1 所示：

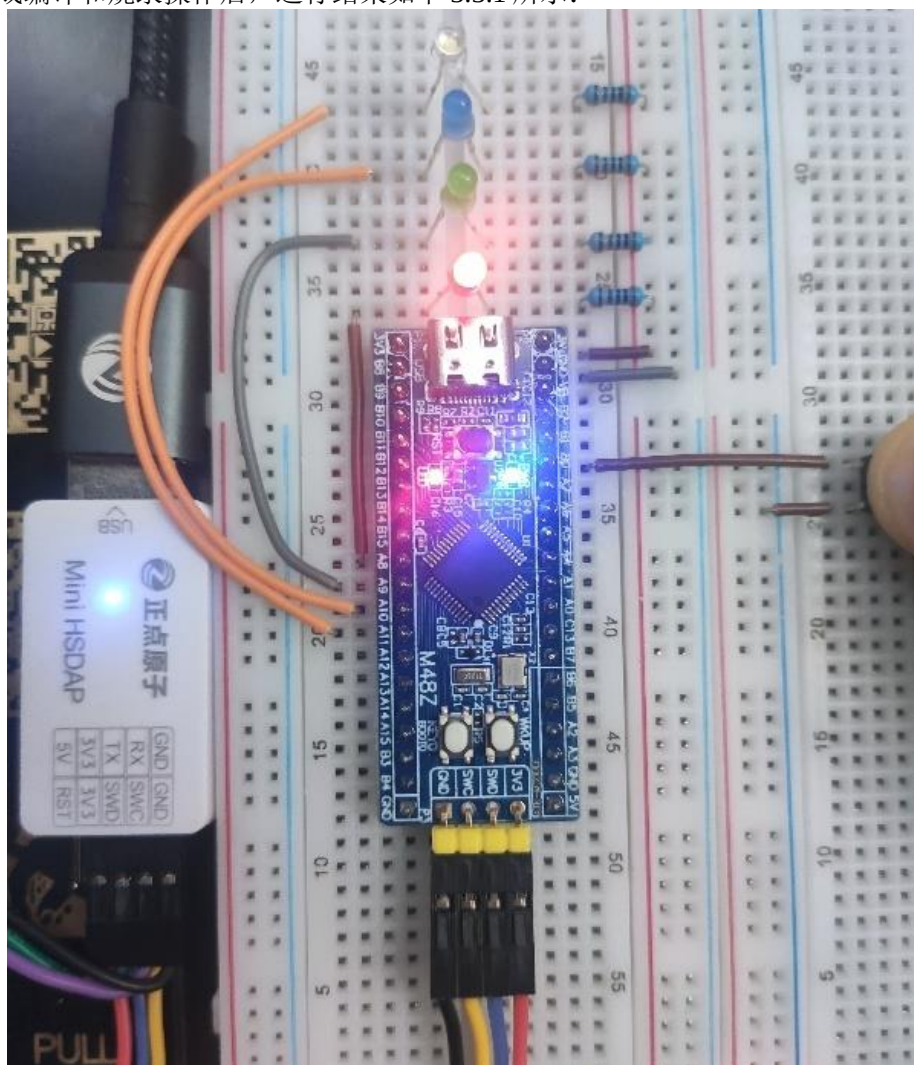


图 3.3.1 程序运行结果

面包板上的 LED 灯（LED0）一开始处于熄灭状态。此时，如果按下 KEY0 按键，可以看到 LED0 被点亮。松开 KEY0 按键，这时候 LED0 会熄灭。这些实验现象符合预期的程序设计效果，说明程序正确性得到了验证。

第四章 多个按键控制 LED 实验

在上一个章节中，我们已经学会了用一个按键控制一个灯的状态，在实际应用中，有可能还需要用到多个按键，所以本章节要使用 4 个按键控制 4 个 LED 灯，继续巩固 GPIO 作为输入功能的知识点。

本章分为如下几个小节：

- 4.1 硬件设计
- 4.2 程序设计
- 4.3 下载验证

4.1 硬件设计

4.1.1 例程功能

通过四个按键去控制四个 LED 灯状态。按下 KEY0，LED0 点亮；按下 KEY1，LED1 点亮；按下 KEY2，LED2 点亮；按下 KEY3，LED3 点亮。

4.1.2 硬件资源

1. LED

LED0 – PA8	LED1 – PA9	LED2 – PA10	LED3 – PA11
------------	------------	-------------	-------------
2. 按键

KEY0 – PB0	KEY1 – PA7	KEY2 – PA6	KEY3 – PA5
------------	------------	------------	------------

4.1.3 原理图

本章相比上一章新增的三个按键（KEY1、KEY2 和 KEY3），这三个 KEY 的连接方式跟 KEY0 是一样的，也是一端连接电源负极，一端连接到 MCU 的 GPIO。

本章设计的电路原理图如下：

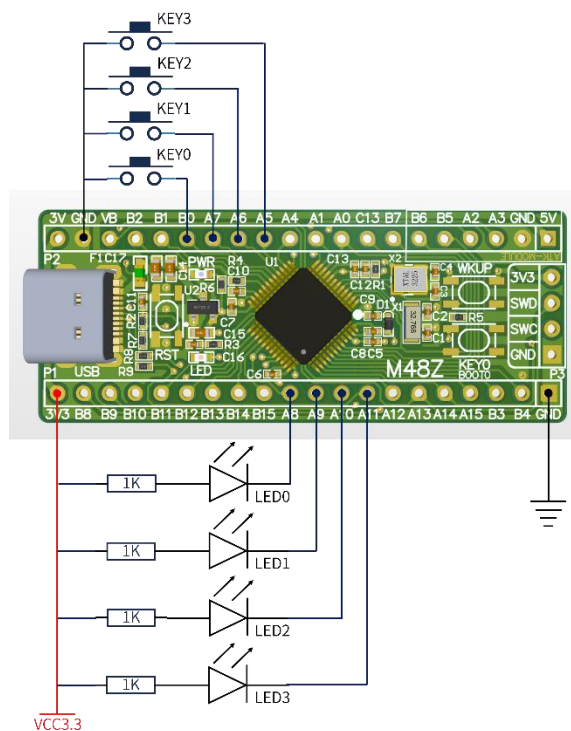


图 4.1.3.1 按键与 MCU 的连接原理图

从上面的原理图中可以看出，四个按键的一端连接到电源负极，而另一端分别连接到 MCU 的 PB0、PA7、PA6 和 PA5 引脚。当按键被按下时，MCU 对应的引脚都能够读取到低电平的状态。当松开按键后，MCU 对应的引脚读取到的电平状态是确定的。

为了读取按键的状态，它们的引脚需要配置为输入模式，并且需要配置成上拉，这样当引脚悬空时，它们会被上拉到电源正极，从而保证引脚电平的准确性。在实际应用中，需要根据电路设计的需求，对按键和 LED 的接线方式进行合理配置。

基于面包板上的实物连接图，可以查看下载验证中的效果图。

4.2 程序设计

本实验使用的 HAL 库 GPIO 函数跟上一个实验是一样的。对比上一个实验只是添加多了 3 个 KEY，没有添加别的东西，所以在 KEY 驱动配置流程也是一样的，配置 GPIO 引脚的模式以及配置 GPIO 引脚输出的电平。

4.2.3 按键驱动代码

本实验的 KEY 驱动会在上一个实验的 KEY 驱动上添加三个 KEY，添加方式也是跟 KEY0 是一致的，这里就不再细说。

在 key.h 文件中，共定义了 4 个 KEY 相关的宏，如下图所示：

```
/* 引脚定义 */
#define KEY0_GPIO_PORT      GPIOB
#define KEY0_GPIO_PIN      GPIO_PIN_0
#define KEY0_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)

#define KEY1_GPIO_PORT      GPIOA
#define KEY1_GPIO_PIN      GPIO_PIN_7
#define KEY1_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define KEY2_GPIO_PORT      GPIOA
#define KEY2_GPIO_PIN      GPIO_PIN_6
#define KEY2_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define KEY3_GPIO_PORT      GPIOA
#define KEY3_GPIO_PIN      GPIO_PIN_5
#define KEY3_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)
```

操作引脚的宏函数也定了 4 个，如下所示：

```
/* I/O 操作 */
#define KEY0      HAL_GPIO_ReadPin(KEY0_GPIO_PORT, KEY0_GPIO_PIN) /* 读取 KEY0 */
#define KEY1      HAL_GPIO_ReadPin(KEY1_GPIO_PORT, KEY1_GPIO_PIN) /* 读取 KEY1 */
#define KEY2      HAL_GPIO_ReadPin(KEY2_GPIO_PORT, KEY2_GPIO_PIN) /* 读取 KEY2 */
#define KEY3      HAL_GPIO_ReadPin(KEY3_GPIO_PORT, KEY3_GPIO_PIN) /* 读取 KEY3 */
```

KEY 的初始化函数除了要初始化 KEY0，还需要初始化 KEY1、KEY2 和 KEY3，如下所示：

```
/**
 * @brief      按键初始化函数
 * @param      无
 * @retval     无
 */
void key_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    KEY0_GPIO_CLK_ENABLE(); /* KEY0 时钟使能 */
    KEY1_GPIO_CLK_ENABLE(); /* KEY1 时钟使能 */
    KEY2_GPIO_CLK_ENABLE(); /* KEY2 时钟使能 */
    KEY3_GPIO_CLK_ENABLE(); /* KEY3 时钟使能 */

    gpio_init_struct.Pin = KEY0_GPIO_PIN; /* KEY0 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_INPUT; /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLUP; /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH; /* 高速 */
}
```

```
HAL_GPIO_Init(KEY0_GPIO_PORT, &gpio_init_struct);

gpio_init_struct.Pin = KEY1_GPIO_PIN;          /* KEY1 引脚 */
HAL_GPIO_Init(KEY1_GPIO_PORT, &gpio_init_struct);

gpio_init_struct.Pin = KEY2_GPIO_PIN;          /* KEY2 引脚 */
HAL_GPIO_Init(KEY2_GPIO_PORT, &gpio_init_struct);

gpio_init_struct.Pin = KEY3_GPIO_PIN;          /* KEY3 引脚 */
HAL_GPIO_Init(KEY3_GPIO_PORT, &gpio_init_struct);
}
```

在按键初始化函数中，对于 4 个按键对应的 GPIO 引脚，开启了其 GPIO 端口时钟，并将其设置为输入模式和上拉模式。

4.2.4 实验应用代码

本章实验的应用代码，如下所示：

```
int main(void)
{
    HAL_Init();          /* 初始化 HAL 库 */
    sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟，72Mhz */
    delay_init(72);      /* 延时初始化 */
    usart_init(115200);  /* 初始化串口 */
    led_init();          /* 初始化 LED */
    key_init();          /* 初始化按键 */

    while (1)
    {
        if (KEY0 == 0)    /* 按键被按下 */
        {
            delay_ms(10); /* 按键消抖 */
            if (KEY0 == 0)
            {
                LED0(0);
            }
        }
        else
        {
            LED0(1);
        }

        if (KEY1 == 0)    /* 按键被按下 */
        {
            delay_ms(10); /* 按键消抖 */
            if (KEY1 == 0)
            {
                LED1(0);
            }
        }
        else
        {
            LED1(1);
        }

        if (KEY2 == 0)    /* 按键被按下 */
        {
            delay_ms(10); /* 按键消抖 */
            if (KEY2 == 0)
            {
                LED2(0);
            }
        }
        else
    }
```

```
{
    LED2(1);
}

if (KEY3 == 0)      /* 按键被按下 */
{
    delay_ms(10);   /* 按键消抖 */
    if (KEY3 == 0)
    {
        LED3(0);
    }
}
else
{
    LED3(1);
}
}
```

在应用代码中，初始化完 LED 和 KEY 后，就进入了一个无限循环中。在循环中来判断按键的状态。如果扫描到某个按键被按下，对应的 LED 就会被点亮，这其中的对应关系为 KEY0 对应 LED0，KEY1 对应 LED1，KEY2 对应 LED2，KEY3 对应 LED3。

4.3 下载验证

在完成编译和烧录操作后，运行结果如下 4.3.1 所示：

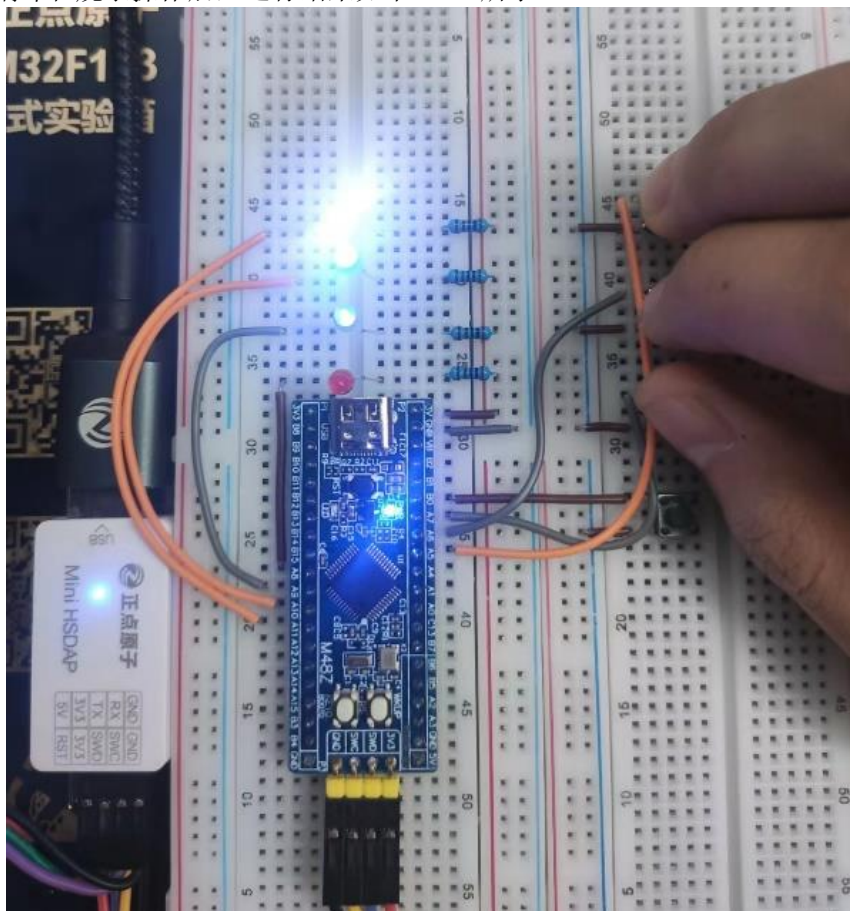


图 4.3.1 程序运行结果

由于按键没有被按下，所以面包板上的 LED 灯都会处于熄灭状态。此时，如果按下 KEY1、KEY2 和 KEY3 按键，这时候可以看到 LED1、LED2 和 LED3 被点亮。KEY0 由于没有被按下，所以 LED0 还是处于熄灭状态。这些实验现象符合预期的程序设计效果，说明程序正确性得到了验证。

第五章 PWM 控制 LED 实验

本章将介绍使用 STM32F103 的定时器输出 PWM。通过本章的学习，读者将学习到如何使用定时器输出比较信号，以及如何使用 PWM 信号控制 LED 或其他负载。至于原理性知识学习，读者可学习正点原子 STM32F103 开发指南即可。

本章分为如下几个小节：

- 5.1 硬件设计
- 5.2 程序设计
- 5.3 下载验证

5.1 硬件设计

5.1.1 例程功能

本实验使用定时器 1 的四个通道输出 PWM，分别作用在 LED0、LED1、LED2 和 LED3，实现 4 个 LED 同时由暗变亮，再由亮变暗，依次循环。

注意：编写程序时，必须把串口初始化函数进行注销，因为本实验使用 PA9 和 PA10 输出 PWM，所以串口初始化函数会影响到这两个引脚输出 PWM。

5.1.2 硬件资源

1. LED
 - LED0 – PA8 LED1 – PA9 LED2 – PA10 LED3 – PA11
2. 定时器 1
 - 通道 1 – PA8 通道 2 – PA9 通道 3 – PA10 通道 4 – PA11

5.1.3 原理图

本章实验使用的定时器 1 为 STM32F103 的片上资源，因此没有对应的连接原理图。硬件上面的连接图与第二章实验是一致的。

5.2 程序设计

5.2.1 HAL 库的 TIM 函数

下面笔者来介绍一下本实验中需要用到的定时器相关函数，如下所示：

1. 配置 TIM

该函数用于配置 TIM，其函数的原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_PWM_Init(TIM_HandleTypeDef *htim)
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 外设结构体的指针

表 5.2.2.1 函数 HAL_TIM_PWM_Init()形参描述

返回值：HAL_StatusTypeDef 枚举类型的值。

该函数使用 TIM_HandleTypeDef 类型的结构体变量传入 TIM 的配置参数。该结构体的定义如下：

```
typedef struct
{
    TIM_TypeDef                *Instance;           /* 外设寄存器基地址 */
    TIM_Base_InitTypeDef       Init;               /* 定时器初始化结构体 */
    HAL_TIM_ActiveChannel      Channel;            /* 定时器通道 */
    DMA_HandleTypeDef          *hdma[7];          /* DMA 管理结构体 */
}
```

```

HAL_LockTypeDef          Lock;          /* 锁定资源 */
__IO HAL_TIM_StateTypeDef State;        /* 定时器状态 */
}TIM_HandleTypeDef;

typedef struct
{
    uint32_t Prescaler;          /* 预分频系数 */
    uint32_t CounterMode;        /* 计数模式 */
    uint32_t Period;             /* 自动重载值 ARR */
    uint32_t ClockDivision;      /* 时钟分频因子 */
    uint32_t RepetitionCounter;   /* 重复计数器 */
    uint32_t AutoReloadPreload;  /* 自动重载预装载使能 */
} TIM_Base_InitTypeDef;

/* CounterMode */
#define TIM_COUNTERMODE_UP           0x00000000U /* 递增计数模式 */
#define TIM_COUNTERMODE_DOWN        TIM_CR1_DIR /* 递减计数模式 */
#define TIM_COUNTERMODE_CENTERALIGNED1 TIM_CR1_CMS_0 /* 中间对齐模式 1 */
#define TIM_COUNTERMODE_CENTERALIGNED2 TIM_CR1_CMS_1 /* 中间对齐模式 2 */
#define TIM_COUNTERMODE_CENTERALIGNED3 TIM_CR1_CMS /* 中间对齐模式 3 */

```

要实现定时器输出 PWM 功能，除了需要配置 Instance 和 Init 这两个成员变量。还需要配置通用定时器的输出比较通道。比较通道配置函数如下：

2. 配置输出比较

该函数用于配置 TIM 的任意输出比较通道，其函数原型如下所示：

```

HAL_StatusTypeDef HAL_TIM_PWM_ConfigChannel(TIM_HandleTypeDef *htim,
                                             TIM_OC_InitTypeDef *sConfig,
                                             uint32_t Channel)

```

该函数的形参描述，如下表所示：

形参	描述
htim	指向 TIM 外设结构体的指针
sConfig	指向输出比较通道配置结构体的指针 需自行定义，根据输出比较通道配置参数填充结构体中的成员
Channel	输出通道 TIM_CHANNEL_1 ~ TIM_CHANNEL_4

表 5.2.1.2 函数 HAL_TIM_PWM_ConfigChannel()形参描述

返回值：HAL_StatusTypeDef 枚举类型的值。

该函数使用 TIM_OC_InitTypeDef 类型的结构体变量传入 TIM 的配置参数。该结构体的定义如下：

```

typedef struct
{
    uint32_t OCMODE;          /* 模式 */
    uint32_t Pulse;           /* 比较值 */
    uint32_t OCPolarity;      /* 极性 */
    uint32_t OCNPolarity;     /* 互补通道极性 */
    uint32_t OCFastMode;      /* 空闲状态 */
    uint32_t OCIdleState;     /* 空闲状态 */
    uint32_t OCNIdleState;    /* 互补通道空闲状态 */
} TIM_OC_InitTypeDef;

/* OCMODE */
#define TIM_OCMODE_TIMING          0x00000000U /* 冻结 */
#define TIM_OCMODE_ACTIVE         TIM_CCMR1_OC1M_0 /* 匹配时输出置为高 */
#define TIM_OCMODE_INACTIVE       TIM_CCMR1_OC1M_1 /* 匹配时输出置为低 */
#define TIM_OCMODE_TOGGLE        (TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1M_0) /* 匹配时输出翻转 */
#define TIM_OCMODE_PWM1          (TIM_CCMR1_OC1M_2 | TIM_CCMR1_OC1M_1) /* PWM 模式 1 */
#define TIM_OCMODE_PWM2          (TIM_CCMR1_OC1M_2 | TIM_CCMR1_OC1M_0)

```

```

|TIM_CCMR1_OC1M_1
|TIM_CCMR1_OC1M_0) /* PWM 模式 2 */

/* OCPolarity */
#define TIM_OCPOLARITY_HIGH 0x00000000U /* 高电平有效 */
#define TIM_OCPOLARITY_LOW TIM_CCER_CC1P /* 低电平有效 */

```

3. 使能捕获比较通道

该函数用于使能捕获比较通道，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel)
```

该函数的形参描述，如下表所示：

形参	描述
htim	指向输出比较通道配置结构体的指针
Channel	输出通道 TIM_CHANNEL_1 ~ TIM_CHANNEL_4

表 5.2.1.3 函数 HAL_TIM_PWM_Start()形参描述

返回值：HAL_StatusTypeDef 枚举类型的值。

4. 配置捕获比较值

在 HAL 库中，配置捕获比较值不再提供函数配置，而是使用宏定义的形式配置，其宏定义原型如下所示：

```
#define __HAL_TIM_SET_COMPARE(__HANDLE__, __CHANNEL__, __COMPARE__)
```

该函数的形参描述，如下表所示：

形参	描述
__HANDLE__	指向 TIM 外设结构体的指针
__CHANNEL__	配置捕获比较值通道
__COMPARE__	比较值配置

表 5.2.1.4 宏定义 __HAL_TIM_SET_COMPARE()形参描述

返回值：无。

5.2.2 TIM 的 PWM 输出驱动配置流程

本章实验旨在使用 TIM1 的 4 个通道都输出 PWM。为此，需要调用 HAL_TIM_PWM_Init 来配置定时器的基本参数，并配置定时器的输出比较通道。具体步骤如下：

1. 开启 TIM1 时钟
2. 开启 GPIOA 时钟，并初始化 PA8、PA9、PA10、PA11 引脚且配置为复用功能
3. 配置 TIM1 的自动重装载值和预分频器数值等参数
4. 配置 4 个输出比较通道，本实验配置 TIM_OC_InitTypeDef 结构体中的 OCMoDe、OCPolarity 和 Pulse 即可
5. 使能 TIM1
6. 使能 TIM1 的 PWM 输出
7. 使能 4 个输出比较通道输出

在 main 函数的 while 循环中同时修改 4 个输出比较通道的捕获计较寄存器数值达到修改 PWM 输出的占空比的目的，从而实现 LED 灯亮度控制。

5.2.3 TIM 输出 PWM 驱动代码

本章实验的定时器驱动主要负责向应用层提供定时器的初始化函数。在本章实验中，定时器的驱动代码包括 tim.c 和 tim.h 两个文件。

定时器驱动中，对 TIM、GPIO 的相关宏定义，如下所示：

```

#define TIM_TIMX_PWM_CH1_GPIO_PORT GPIOA
#define TIM_TIMX_PWM_CH1_GPIO_PIN GPIO_PIN_8
#define TIM_TIMX_PWM_CH1_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define TIM_TIMX_PWM_CH2_GPIO_PORT GPIOA

```

```
#define TIM_TIMX_PWM_CH2_GPIO_PIN      GPIO_PIN_9
#define TIM_TIMX_PWM_CH2_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define TIM_TIMX_PWM_CH3_GPIO_PORT      GPIOA
#define TIM_TIMX_PWM_CH3_GPIO_PIN      GPIO_PIN_10
#define TIM_TIMX_PWM_CH3_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define TIM_TIMX_PWM_CH4_GPIO_PORT      GPIOA
#define TIM_TIMX_PWM_CH4_GPIO_PIN      GPIO_PIN_11
#define TIM_TIMX_PWM_CH4_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define TIM_TIMX_PWM                    TIM1
#define TIM_TIMX_PWM_CH1                TIM_CHANNEL_1
#define TIM_TIMX_PWM_CH2                TIM_CHANNEL_2
#define TIM_TIMX_PWM_CH3                TIM_CHANNEL_3
#define TIM_TIMX_PWM_CH4                TIM_CHANNEL_4
#define TIM_TIMX_PWM_CHY_CLK_ENABLE()
do{ __HAL_RCC_TIM1_CLK_ENABLE(); }while(0)    /* TIM1 时钟使能 */
```

定时器驱动中 TIM1 的初始化函数，如下所示：

```
TIM_HandleTypeDef g_timx_pwm_chy_handle;    /* 定时器 x 句柄 */

/**
 * @brief      定时器 TIMX 通道 Y PWM 输出 初始化函数（使用 PWM 模式 1）
 * @note
 *
 *      定时器的时钟来自 APB1, 当 PPRE1 ≥ 2 分频的时候
 *      定时器的时钟为 APB1 时钟的 2 倍，而 APB1 为 36M，所以定时器时钟 = 72Mhz
 *      定时器溢出时间计算方法：Tout = ((arr + 1) * (psc + 1)) / Ft us.
 *      Ft=定时器工作频率,单位:Mhz
 *
 * @param      arr: 自动重装值。
 * @param      psc: 时钟预分频数
 * @retval     无
 */
void tim_timx_pwm_chy_init(uint16_t arr, uint16_t psc)
{
    TIM_OC_InitTypeDef timx_oc_pwm_chy = {0};    /* 定时器 PWM 输出配置 */

    g_timx_pwm_chy_handle.Instance = TIM_TIMX_PWM;    /* 定时器 */
    g_timx_pwm_chy_handle.Init.Prescaler = psc;    /* 定时器分频 */
    /* 递增计数模式 */
    g_timx_pwm_chy_handle.Init.CounterMode = TIM_COUNTERMODE_UP;
    g_timx_pwm_chy_handle.Init.Period = arr;    /* 自动重装载值 */
    HAL_TIM_PWM_Init(&g_timx_pwm_chy_handle);    /* 初始化 PWM */

    timx_oc_pwm_chy.OCMode = TIM_OCMode_PWM1;    /* 模式选择 PWM1 */
    timx_oc_pwm_chy.Pulse = arr / 2;    /* 设置比较值,此值用来确定占空比 */
    /* 这里默认设置比较值为自动重装载值的一半,即占空比为 50% */
    timx_oc_pwm_chy.OCpolarity = TIM_OCPolarity_LOW;    /* 输出比较极性为低 */
    HAL_TIM_PWM_ConfigChannel(&g_timx_pwm_chy_handle,
                             &timx_oc_pwm_chy,
                             TIM_TIMX_PWM_CH1);    /* 配置 TIMx 通道 1 */

    /* 开启对应 PWM 通道 */
    HAL_TIM_PWM_Start(&g_timx_pwm_chy_handle, TIM_TIMX_PWM_CH1);

    HAL_TIM_PWM_ConfigChannel(&g_timx_pwm_chy_handle,
                             &timx_oc_pwm_chy,
                             TIM_TIMX_PWM_CH2);    /* 配置 TIMx 通道 2 */

    /* 开启对应 PWM 通道 */
    HAL_TIM_PWM_Start(&g_timx_pwm_chy_handle, TIM_TIMX_PWM_CH2);
```

```

HAL_TIM_PWM_ConfigChannel(&g_timx_pwm_chy_handle,
                          &timx_oc_pwm_chy,
                          TIM_TIMX_PWM_CH3);          /* 配置 TIMx 通道 3 */

/* 开启对应 PWM 通道 */
HAL_TIM_PWM_Start(&g_timx_pwm_chy_handle, TIM_TIMX_PWM_CH3);

HAL_TIM_PWM_ConfigChannel(&g_timx_pwm_chy_handle,
                          &timx_oc_pwm_chy,
                          TIM_TIMX_PWM_CH4);          /* 配置 TIMx 通道 4 */

/* 开启对应 PWM 通道 */
HAL_TIM_PWM_Start(&g_timx_pwm_chy_handle, TIM_TIMX_PWM_CH4);
}

/**
 * @brief      定时器底层驱动，时钟使能，引脚配置
 *             此函数会被 HAL_TIM_PWM_Init() 调用
 * @param      htim: 定时器句柄
 * @retval      无
 */
void HAL_TIM_PWM_MspInit(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == GTIM_TIMX_PWM)
    {
        GPIO_InitTypeDef gpio_init_struct;
        TIM_TIMX_PWM_CH1_GPIO_CLK_ENABLE();          /* 开启通道 1 的 GPIO 时钟 */
        TIM_TIMX_PWM_CH2_GPIO_CLK_ENABLE();          /* 开启通道 2 的 GPIO 时钟 */
        TIM_TIMX_PWM_CH3_GPIO_CLK_ENABLE();          /* 开启通道 3 的 GPIO 时钟 */
        TIM_TIMX_PWM_CH4_GPIO_CLK_ENABLE();          /* 开启通道 4 的 GPIO 时钟 */
        TIM_TIMX_PWM_CHY_CLK_ENABLE();

        gpio_init_struct.Pin = TIM_TIMX_PWM_CH1_GPIO_PIN; /* 通道 1 的 GPIO 口 */
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;          /* 复用推挽输出 */
        gpio_init_struct.Pull = GPIO_PULLUP;             /* 上拉 */
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;   /* 高速 */
        HAL_GPIO_Init(TIM_TIMX_PWM_CH1_GPIO_PORT, &gpio_init_struct);

        gpio_init_struct.Pin = TIM_TIMX_PWM_CH2_GPIO_PIN; /* 通道 2 的 GPIO 口 */
        HAL_GPIO_Init(TIM_TIMX_PWM_CH2_GPIO_PORT, &gpio_init_struct);

        gpio_init_struct.Pin = TIM_TIMX_PWM_CH3_GPIO_PIN; /* 通道 3 的 GPIO 口 */
        HAL_GPIO_Init(TIM_TIMX_PWM_CH3_GPIO_PORT, &gpio_init_struct);

        gpio_init_struct.Pin = TIM_TIMX_PWM_CH4_GPIO_PIN; /* 通道 4 的 GPIO 口 */
        HAL_GPIO_Init(TIM_TIMX_PWM_CH4_GPIO_PORT, &gpio_init_struct);
    }
}

```

从 TIM1 的初始化代码中可以看到，不仅配置了 TIM1 的自动重装载值和预分频器数值等基本参数，还配置了 TIM1 的 4 个输出比较通道。同时，由于需要使用 GPIO 引脚输出 PWM，因此对应的 GPIO 引脚也配置了复用功能。

5.2.4 实验应用代码

本章实验的应用代码，如下所示：

```

int main(void)
{
    uint32_t ledpwmval = 0;
    uint8_t dir = 1;

    HAL_Init();          /* 初始化 HAL 库 */
    sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟，72Mhz */
    delay_init(72);      /* 延时初始化 */
}

```

```
//usart_init(115200); /* 初始化串口 */
tim_timx_pwm_chy_init(500 - 1, 72 - 1); /* 1Mhz 的计数频率, 2Khz 的 PWM */

while (1)
{
    delay_ms(10);

    /* 根据方向修改 ledpwmval */
    if (dir == 1)
    {
        ledpwmval++;
    }
    else
    {
        ledpwmval--;
    }

    /* 当 ledpwmval 大于 300 时, 方向改为递减 */
    if (ledpwmval > 300)
    {
        dir = 0;
    }

    /* 当 ledpwmval 等于 0 时, 方向改为递增 */
    if (ledpwmval == 0)
    {
        dir = 1;
    }

    /* 修改比较值控制占空比 */
    __HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle,
                          TIM_TIMX_PWM_CH1, ledpwmval);

    __HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle,
                          TIM_TIMX_PWM_CH2, ledpwmval);

    __HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle,
                          TIM_TIMX_PWM_CH3, ledpwmval);

    __HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle,
                          TIM_TIMX_PWM_CH4, ledpwmval);
}
}
```

从上面的代码中可以看到, 在初始化完 TIM1 输出 PWM 后, 程序就开始不断地改变 TIM1 的 4 个通道的比较值, 以达到改变 PWM 占空比的目的, 使得 LED 灯的亮度也会随之改变, 从而实现了呼吸灯的效果。

5.3 下载验证

在完成编译和烧录操作后, 运行结果如下 5.3.1 所示:

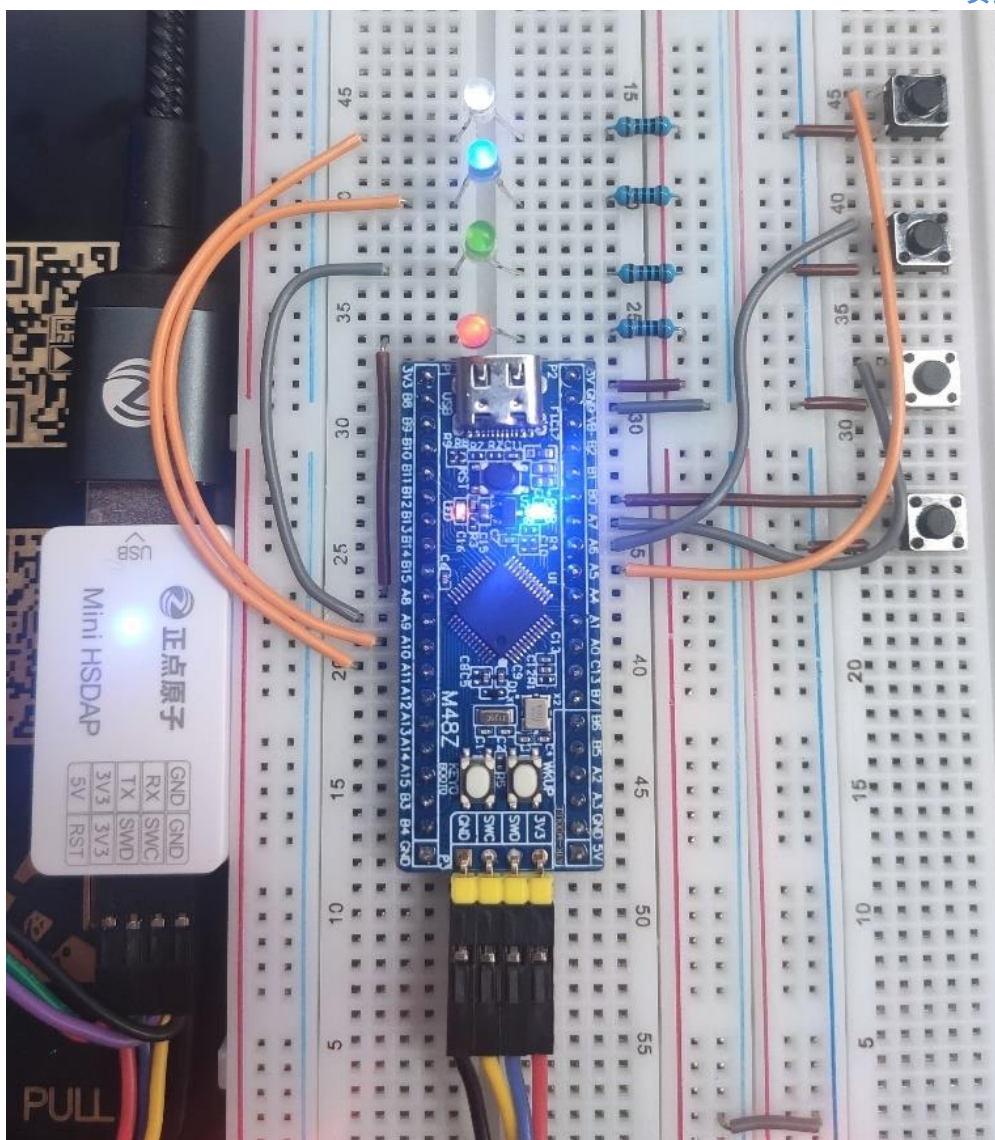


图 5.3.1 程序运行结果

可以看到开发板上的 LED0、LED1、LED2 和 LED3 先由暗逐渐变亮，再由亮逐渐变暗，依次循环，实现了呼吸灯的效果。

第六章 可调亮度 LED 实验

本章将介绍如何使用 STM2F103 中的 ADC 进行单通道的电压采集。通过本章的学习，读者将了解如何使用单通道 ADC 进行电压采集，并掌握 ADC 的基本原理和使用方法。

本章分为如下几个小节：

- 6.1 硬件设计
- 6.2 程序设计
- 6.3 下载验证

6.1 硬件设计

6.1.1 例程功能

1. ADC 采集通道 1 (PA1) 上的电压，PA1 连接到的是一个电位器。通过扭动电位器，采样到不同的 AD 值，该值设置为 TIM1 的 4 个通道的比较值，最终使得四个 LED 灯亮度发生变化。

注意：编写程序时，必须把串口初始化函数进行注销，因为本实验使用 PA9 和 PA10 输出 PWM，所以串口初始化函数会影响到这两个引脚输出 PWM。

6.1.2 硬件资源

- 1. LED
 - LED0 – PA8 LED1 – PA9 LED2 – PA10 LED3 – PA11
- 2. 定时器 1
 - 通道 1 – PA8 通道 2 – PA9 通道 3 – PA10 通道 4 – PA11
- 3. ADC
 - 通道 1 – PA1 (怎么知道 PA1 对应 ADC 的通道 1。请见 STM32F103 数据手册)

6.1.3 原理图

本章实验使用到 ADC 通道 1 在芯片内部已经连接到 PA1 引脚，在外部一个电位器连接到 PA1。当扭动电位器会有不同的电压值输出，此时 ADC 通道 1 即可检测到。电位器实物图如下图所示。



图 6.1.3.1 电位器实物图

配件包的电位器为 5K Ω ，中间的引脚接 AD 引脚，而两边的引脚接电源引脚即可，使用简单。

本章设计的电路原理图如下：

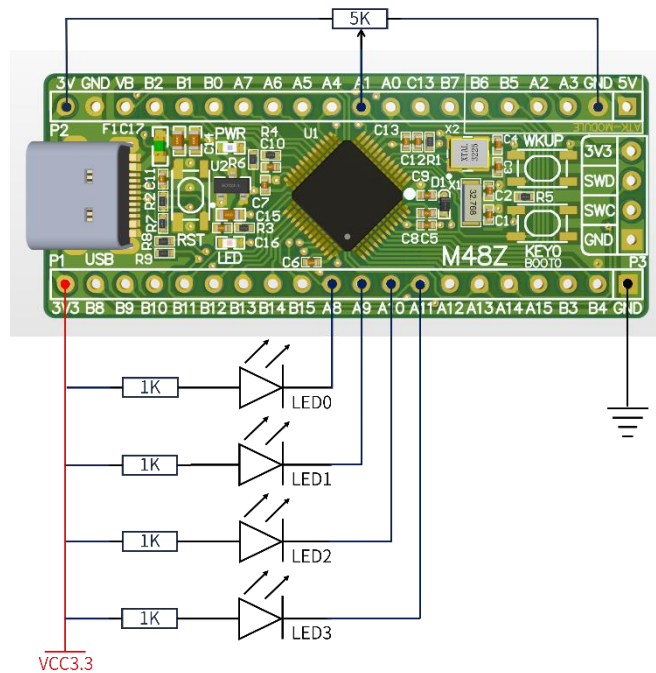


图 6.1.3.2 电位器实物图

基于面包板上的实物连接图，可以查看下载验证中的效果图。

6.2 程序设计

6.2.1 HAL 库 ADC 驱动函数

HAL 库提供了用于操作 STM32F103 片上 ADC 的各种 API 函数。其中包括了配置 ADC、使能 ADC 等函数。本实验使用的 ADC 函数如下：

1. 配置 ADC

该函数用于配置 ADC，其函数原型如下所示：

```
HAL_StatusTypeDef HAL_ADC_Init(ADC_HandleTypeDef* hadc)
```

该函数的形参描述，如下表所示：

形参	描述
hadc	指向 ADC 初始化结构体的指针 需自行定义，并根据 ADC 的配置参数填充结构体中的成员

表 6.2.1.1 函数 HAL_ADC_Init()形参描述

返回值：HAL_StatusTypeDef 枚举类型的值。

该函数使用 ADC_HandleTypeDef 类型的结构体变量传入 ADC 的配置参数。该结构体的定义如下所示：

```
typedef struct
{
    ADC_TypeDef                *Instance;           /* ADC 寄存器基地址 */
    ADC_InitTypeDef            Init;                /* ADC 参数初始化结构体变量 */
    DMA_HandleTypeDef          *DMA_Handle;         /* DMA 配置结构体 */
    HAL_LockTypeDef             Lock;                /* ADC 锁定对象 */
    __IO uint32_t               State;               /* ADC 工作状态 */
    __IO uint32_t               ErrorCode;           /* ADC 错误代码 */
}ADC_HandleTypeDef;
```

该结构体定义和其他外设比较类似，我们着重看第二个成员变量 Init 含义，它是结构体 ADC_InitTypeDef 类型，结构体 ADC_InitTypeDef 定义为：

```
typedef struct {
    uint32_t DataAlign;           /* 设置数据的对齐方式 */
    uint32_t ScanConvMode;        /* 扫描模式 */
}
```

```
FunctionalState ContinuousConvMode; /* 开启连续转换模式否则就是单次转换模式 */
uint32_t NbrOfConversion; /* 设置转换通道数目 */
FunctionalState DiscontinuousConvMode; /* 是否使用规则通道组间断模式 */
uint32_t NbrOfDiscConversion; /* 配置间断模式的规则通道个数 */
uint32_t ExternalTrigConv; /* ADC 外部触发源选择 */
} ADC_InitTypeDef;
```

1) DataAlign: 用于设置数据的对齐方式, 这里可以选择右对齐或者是左对齐, 该参数可选为: ADC_DATAALIGN_RIGHT 和 ADC_DATAALIGN_LEFT。

2) ScanConvMode: 配置是否使用扫描。如果是单通道转换使用 ADC_SCAN_DISABLE, 如果是多通道转换使用 ADC_SCAN_ENABLE。

3) ContinuousConvMode: 可选参数为 ENABLE 和 DISABLE, 配置自动连续转换还是单次转换。使用 ENABLE 配置为使能自动连续转换; 使用 DISABLE 配置为单次转换, 转换一次后停止需要手动控制才重新启动转换。

4) NbrOfConversion: 指定规则组转换通道数目, 范围是: 1~16。

5) DiscontinuousConvMode: 配置是否使用规则通道组间断模式, 比如要转换的通道有 1、2、5、7、8、9, 那么第一次触发会进行通道 1 和 2, 下次触发就是转换通道 5 和 7, 这样不连续的转换, 依次类推。此参数只有将 ScanConvMode 使能, 还有 ContinuousConvMode 失能的情况下才有效, 不可同时使能。

6) NbrOfDiscConversion: 配置间断模式的通道个数, 禁止规则通道组间断模式后, 此参数忽略。

7) ExternalTrigConv: 外部触发方式的选择, 如果使用软件触发, 那么外部触发会关闭。

2. ADC 校准

该函数用于 ADC 校准, 其函数原型如下所示:

```
HAL_StatusTypeDef HAL_ADCEx_Calibration_Start(ADC_HandleTypeDef *hadc)
```

该函数的形参描述, 如下表所示:

形参	描述
hadc	指向 ADC 初始化结构体的指针

表 6.2.1.2 函数 HAL_ADCEx_Calibration_Start()形参描述

返回值: SET: 校准成功; RESET: 校准失败。

3. 配置 ADC 通道

该函数用于配置 ADC 通道, 其函数原型如下所示:

```
HAL_StatusTypeDef HAL_ADC_ConfigChannel(ADC_HandleTypeDef *hadc,
                                         ADC_ChannelConfTypeDef *sConfig)
```

该函数的形参描述, 如下表所示:

形参	描述
hadc	指向 ADC 初始化结构体的指针
sConfig	指向 ADC_ChannelConfTypeDef 结构体类型指针

表 6.2.1.3 函数 HAL_ADC_ConfigChannel()形参描述

返回值: HAL_StatusTypeDef 枚举类型的值。

该函数使用 ADC_ChannelConfTypeDef 类型的结构体变量传入 ADC 的配置参数。该结构体的定义如下所示:

```
typedef struct {
    uint32_t Channel; /* ADC 转换通道 */
    uint32_t Rank; /* ADC 转换顺序 */
    uint32_t SamplingTime; /* ADC 采样周期 */
} ADC_ChannelConfTypeDef;
```

1) Channel: ADC 转换通道, 范围: 0~17。

2) Rank: 在规则转换中的规则组的转换顺序, 可以选择 1~16。

3) SamplingTime: ADC 的采样周期, 最大 239.5 个 ADC 时钟周期, 要求尽量大以减少误差。

4. 开启 ADC 转换

该函数用于配置指定开启 ADC 转换, 其函数原型如下所示:

```
HAL_StatusTypeDef HAL_ADC_Start(ADC_HandleTypeDef *hadc)
```

该函数的形参描述，如下表所示：

形参	描述
hadc	指向 ADC 初始化结构体的指针

表 6.2.1.4 函数 HAL_ADC_Start()形参描述

返回值：HAL_StatusTypeDef 枚举类型的值。

5. 获取 ADC 转换值

该函数用于获取 ADC 转换值，其函数原型如下所示：

```
uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef *hadc)
```

该函数的形参描述，如下表所示：

形参	描述
hadc	指向 ADC 初始化结构体的指针

表 6.2.1.5 函数 HAL_ADC_GetValue()形参描述

返回值：ADC 转换值。

6.2.2 ADC 配置流程

本章实验将使用 ADC 的通道 1（PA1 引脚）采集外部输入电压的模拟量，并将其转换为数字量。具体步骤如下：

1. 开启 ADC 和 GPIOA 时钟
2. PA1 配置模拟输入模式
3. 配置 ADC，如扫描模式、触发检测、数据对齐等
4. 使能 ADC
5. 校验 ADC
6. 启动转换规则通道
7. 等待规则通道转换结束
8. 读取规则通道的转换结果

6.2.3 ADC 驱动代码

本章实验涉及 ADC 驱动的开发，ADC 驱动主要负责向应用层提供 ADC 的初始化和获取 ADC 转换结果的函数。ADC 的驱动代码包括 adc.c 和 adc.h 两个文件。在 adc.c 文件中，主要实现了 ADC 的初始化函数，以及获取 ADC 转换函数。在 adc.h 文件中，主要包含了 ADC 的寄存器定义以及 GPIO 的定义。

ADC 驱动中，对 ADC、GPIO 的相关宏定义，如下所示：

```
/* 单通道 ADC 采集定义 */
#define ADC_ADCX_CHY_GPIO_PORT          GPIOA
#define ADC_ADCX_CHY_GPIO_PIN           GPIO_PIN_1
#define ADC_ADCX_CHY_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)          /* PA 口时钟使能 */

#define ADC_ADCX                          ADC1
#define ADC_ADCX_CHY                      ADC_CHANNEL_1
#define ADC_ADCX_CHY_CLK_ENABLE()
do{ __HAL_RCC_ADC1_CLK_ENABLE(); }while(0)          /* ADC1 时钟使能 */
```

ADC 驱动中，ADC 的初始化函数，如下所示：

```
ADC_HandleTypeDef g_adc_handle;    /* ADC 句柄 */

/**
 * @brief      ADC 初始化函数
 * @note      本函数支持 ADC1/ADC2 任意通道，但是不支持 ADC3
 *            我们使用 12 位精度，ADC 采样时钟=12M，转换时间为：
 *            采样周期 + 12.5 个 ADC 周期
 *            设置最大采样周期：239.5，则转换时间 = 252 个 ADC 周期 = 21us
 * @param      无
 * @retval     无
```

```

*/
void adc_init(void)
{
    g_adc_handle.Instance = ADC_ADCX; /* 选择哪个 ADC */
    g_adc_handle.Init.DataAlign = ADC_DATAALIGN_RIGHT; /* 数据对齐方式：右对齐 */
    /* 非扫描模式，仅用到一个通道 */
    g_adc_handle.Init.ScanConvMode = ADC_SCAN_DISABLE;
    g_adc_handle.Init.ContinuousConvMode = DISABLE; /* 关闭连续转换模式 */
    /* 赋值范围是 1~16，本实验用到 1 个规则通道序列 */
    g_adc_handle.Init.NbrOfConversion = 1;
    g_adc_handle.Init.DiscontinuousConvMode = DISABLE; /* 禁止规则通道组间断模式 */
    /* 配置间断模式的规则通道个数，禁止规则通道组间断模式后，此参数忽略 */
    g_adc_handle.Init.NbrOfDiscConversion = 0;
    /* 触发转换方式：软件触发 */
    g_adc_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    HAL_ADC_Init(&g_adc_handle); /* 初始化 */

    HAL_ADCEx_Calibration_Start(&g_adc_handle); /* 校准 ADC */
}

```

该函数主要调用了两个 HAL 库函数，HAL_ADC_Init 函数配置了选择哪个 ADC、数据对齐方式、是否使用扫描模式等参数，HAL_ADCEx_Calibration_Start 函数用于校准 ADC。另外 HAL_ADC_Init 函数会调用它的 MSP 回调函数 HAL_ADC_MspInit，该函数用来存放使能 ADC 和通道对应 IO 的时钟和初始化 IO 口等代码，其定义如下：

```

/**
 * @brief      ADC 底层驱动，引脚配置，时钟使能
 *              此函数会被 HAL_ADC_Init() 调用
 * @param      hadc: ADC 句柄
 * @retval     无
 */
void HAL_ADC_MspInit(ADC_HandleTypeDef *hadc)
{
    if(hadc->Instance == ADC_ADCX)
    {
        GPIO_InitTypeDef gpio_init_struct;
        RCC_PeriphCLKInitTypeDef adc_clk_init = {0};

        ADC_ADCX_CHY_CLK_ENABLE(); /* 使能 ADCx 时钟 */
        ADC_ADCX_CHY_GPIO_CLK_ENABLE(); /* 开启 GPIO 时钟 */

        /* 设置 ADC 时钟 */
        adc_clk_init.PeriphClockSelection = RCC_PERIPHCLK_ADC; /* ADC 外设时钟 */
        /* 分频因子 6 时钟为 72M/6=12MHz */
        adc_clk_init.AdcClockSelection = RCC_ADCPCLK2_DIV6;
        HAL_RCCEx_PeriphCLKConfig(&adc_clk_init); /* 设置 ADC 时钟 */

        /* 设置 AD 采集通道对应 IO 引脚工作模式 */
        gpio_init_struct.Pin = ADC_ADCX_CHY_GPIO_PIN; /* ADC 通道 IO 引脚 */
        gpio_init_struct.Mode = GPIO_MODE_ANALOG; /* 模拟 */
        HAL_GPIO_Init(ADC_ADCX_CHY_GPIO_PORT, &gpio_init_struct);
    }
}

```

可以看到在 HAL_ADC_MspInit 函数中，我们除了使能 ADC 和通道对应 IO 时钟、初始化 IO 外，还配置了 ADC 的时钟预分频系数。ADC 的时钟源是 PCLK2（72MHz），经过 6 分频后，得到 ADC 的输入时钟是 12MHz

ADC 驱动中，获取 ADC 转换结果的函数，如下所示：

```

/**
 * @brief      获得 ADC 转换后的结果
 * @param      ch: 通道值 0~17，取值范围为：ADC_CHANNEL_0~ADC_CHANNEL_17
 * @retval     无
 */

```

```
uint32_t adc_get_result(uint32_t ch)
{
    /* 设置通道, 序列和采样时间 */
    adc_channel_set(&g_adc_handle, ch,
                    ADC_REGULAR_RANK_1, ADC_SAMPLETIME_239CYCLES_5);

    HAL_ADC_Start(&g_adc_handle); /* 开启 ADC */
    HAL_ADC_PollForConversion(&g_adc_handle, 10); /* 轮询转换 */
    /* 返回最近一次 ADC1 规则组的转换结果 */
    return (uint16_t)HAL_ADC_GetValue(&g_adc_handle);
}

/**
 * @brief      获取通道 ch 的转换值, 取 times 次, 然后平均
 * @param      ch      : 通道号, 0~17
 * @param      times   : 获取次数
 * @retval     通道 ch 的 times 次转换结果平均值
 */
uint32_t adc_get_result_average(uint32_t ch, uint8_t times)
{
    uint32_t temp_val = 0;
    uint8_t t;

    for (t = 0; t < times; t++) /* 获取 times 次数据 */
    {
        temp_val += adc_get_result(ch);
        delay_ms(5);
    }

    return temp_val / times; /* 返回平均值 */
}
```

以上两个函数都是用于获取 ADC 转换结果的函数。其中, 函数 `adc_get_result` 会配置并开启 ADC 指定通道的规则通道转换, 并等待其转换结束后, 读取其转换的一次结果。而函数 `adc_get_result_average` 则是多次调用 `adc_get_result` 函数获取多次 ADC 的转换结果, 然后进行均值滤波。

6.2.4 实验应用代码

本章实验的应用代码, 如下所示:

```
int main(void)
{
    uint16_t adcddata;
    uint16_t pwm_value;

    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟, 72Mhz */
    delay_init(72); /* 延时初始化 */
    //usart_init(115200); /* 初始化串口 */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化 KEY */
    tim_timx_pwm_chy_init(500 - 1, 72 - 1); /* 1Mhz 的计数频率, 2Khz 的 PWM */
    adc_init(); /* 初始化 ADC */

    while (1)
    {
        /* 获取 ADC 通道 10 次转换且进行均值滤波后的结果 */
        adcddata = adc_get_result_average(ADC_ADCX_CHY, 10);
        pwm_value = (adcddata * 500) / 4095;

        /* 修改比较值控制占空比 */
        __HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle,
                               TIM_TIMX_PWM_CH1, pwm_value);
    }
}
```

```
__HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle,  
                      TIM_TIMX_PWM_CH2, pwm_value);  
  
__HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle,  
                      TIM_TIMX_PWM_CH3, pwm_value);  
  
__HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle,  
                      TIM_TIMX_PWM_CH4, pwm_value);  
  
LED0_TOGGLE();  
delay_ms(100);  
}  
}
```

在上面的代码中，完成了 ADC 的初始化工作之后，程序开始不断地获取 ADC 通道 1 的 10 次转换结果，并对这些结果进行均值滤波。最终把得到的 AD 值转换为自动重装值，作用在定时器 1 的四个通道中，实现电位器控制 LED 亮度效果。

6.3 下载验证

在完成编译和烧录操作后，运行结果如下图所示。

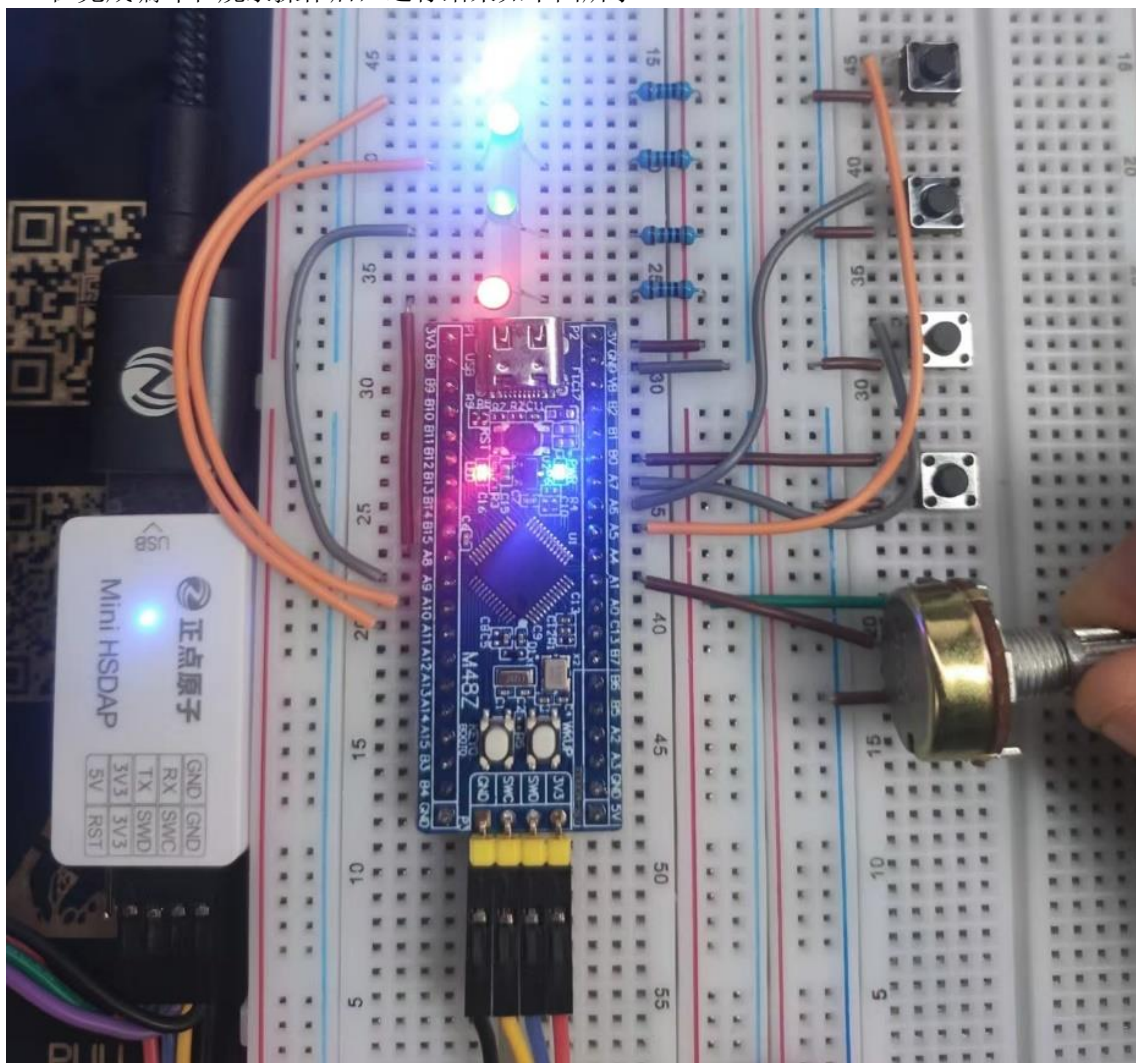


图 6.3.1 程序运行结果

顺时针和逆时针都会改变电压值，最终导致 ADC 通道 1 采样到的值发生改变，导致 LED 的亮度从暗变亮或从亮变暗。

需要注意的是，输入电压不能超过开发板的 3.3V 阈值，否则可能会损坏开发板。